



Speech Processing

Module 5

The Bayesian Method for Pronunciation

- used to pick the optimal correction for a spelling error
- used to solve the pronunciation subproblem in speech recognition.
- we are given a series of phones and our job is to compute the most probable word which generated them.
- *Consider the particular problem of interpreting the sequence of phones*
- *[ni] , when it occurs after the word I at the beginning of a sentence*
-

- 
- *The seven words are the, neat, need, new, knee, to, and you.*
 - *The pronunciation of **the** as [ni] is caused by the regressive assimilation process*
 - ***the** pronounced as [ni] in Switchboard occurred after words like in, on, and been (so in the ! [Inni]).*
 - *The pronunciation of **new** as [ni] occurred most frequently in the word **New York**; the vowel [u] has fronted to [i] before a [y]*
 - *The pronunciation of **to** as [ni] occurred after the work **talking** (talking to you ! [tOklniyu])*
 - *Consider only the following five words as candidate lexical forms for [ni] : knee, the, neat, need, new.*

The Bayesian Method

- choosing the word whose product of prior and likelihood is the highest,
- where
- y represents the sequence of phones (in this case [ni])
- w represents the candidate word (the, new, etc)).

$$\hat{w} = \operatorname{argmax}_{w \in W} \underbrace{P(y|w)}_{\text{likelihood}} \underbrace{P(w)}_{\text{prior}}$$

- Generate pronunciation likelihoods is via probabilistic rules.

- a word-initial [D] becomes [n] if the preceding word ended in [n] or sometimes [m] :

- Let n_{count} be the number of times lexical [D] is realized word-initially by surface [n] when the previous word ends in a nasal
- Let env_{count} be the total number of times lexical [D] occurs (whatever its surface realization) when the previous word ends in a nasal

Word	Rule Name	Rule	P
<i>the</i>	nasal assimilation	$\partial \Rightarrow n / [+nasal] \# ___$	[.15]
<i>neat</i>	final t deletion	$t \Rightarrow \emptyset / V ___ \#$	[.52]
<i>need</i>	final d deletion	$d \Rightarrow \emptyset / V ___ \#$	[.11]
<i>new</i>	u fronting	$u \Rightarrow i / ___ \# [y]$	[.36]

Figure 5.10 Simple rules of pronunciation variation due to context in continuous speech accounting for the pronunciation of each of these words as [ni].

w	freq(w)	p(w)
knee	61	.000024
the	114,834	.046
neat	338	.00013
need	1417	.00056
new	2625	.001

Word	p(y w)	p(w)	p(y w)p(w)
<i>new</i>	.36	.001	.00036
<i>neat</i>	.52	.00013	.000068
<i>need</i>	.11	.00056	.000062
<i>knee</i>	1.00	.000024	.000024
<i>the</i>	0	.046	0



Decision tree models

- Classification and Regression Tree (CART)
- Automatically inducing lexical-to-surface
- pronunciations mappings from a labeled corpus
- DT takes a situation described by a set of features and classifies it into a category and an associated probability.
- For pronunciation, a decision tree can be trained to take a lexical phone and various contextual features (surrounding phones, stress and syllable structure information, lexical identity) and select an appropriate surface phone to realize it

-

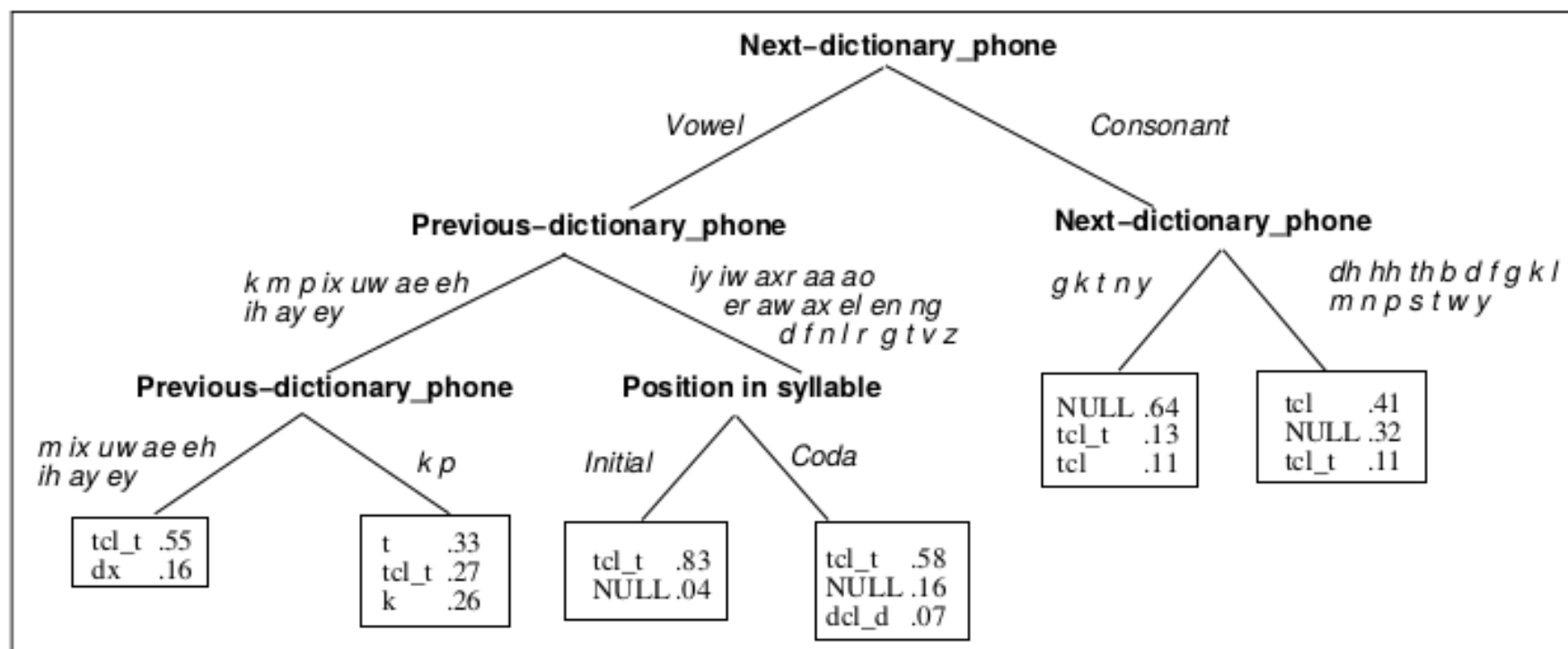


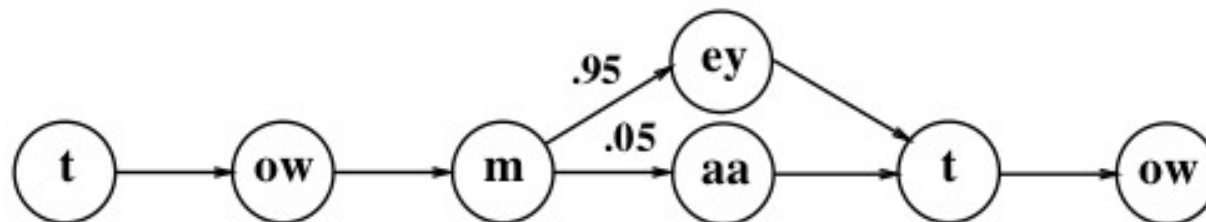
Figure 5.11 Hand-pruned decision tree for the phoneme /t/ induced from the Switchboard corpus (courtesy of Eric Fosler-Lussier). This particular decision tree doesn't model flapping since flaps were already listed in the dictionary. The tree automatically induced the categories *Vowel* and *Consonant*. We have only shown the most likely realizations at each leaf node.

Weighted Automata

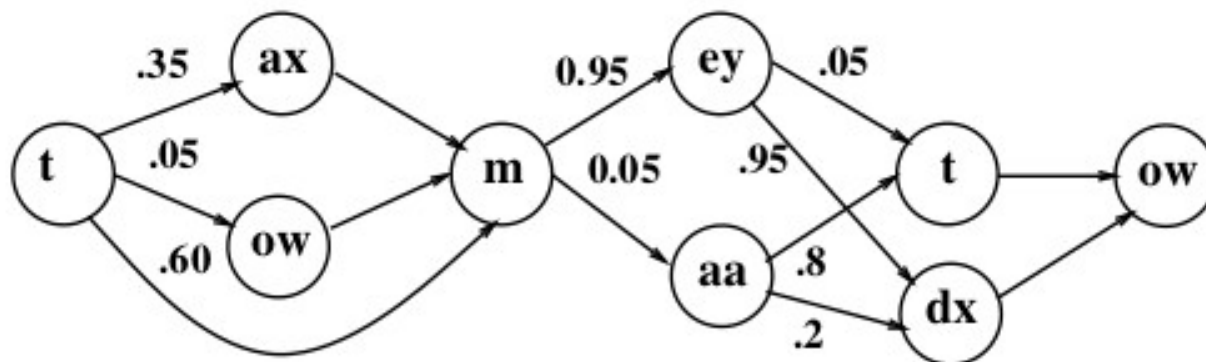
- A simple augmentation of the finite automaton in which each arc is associated with a probability, indicating how likely that path is to be taken
- Probability on all the arcs leaving a node must sum to 1
- A Markov chain is a special case of a weighted automaton
- input sequence uniquely determines which states the automaton will go through
- can't represent ambiguous problems
- Typewriting spelling correction – Weighted automata
- Handwriting spelling correction - HMM

Weighted Automata

Word model with dialect variation:



Word model with coarticulation and dialect variation:

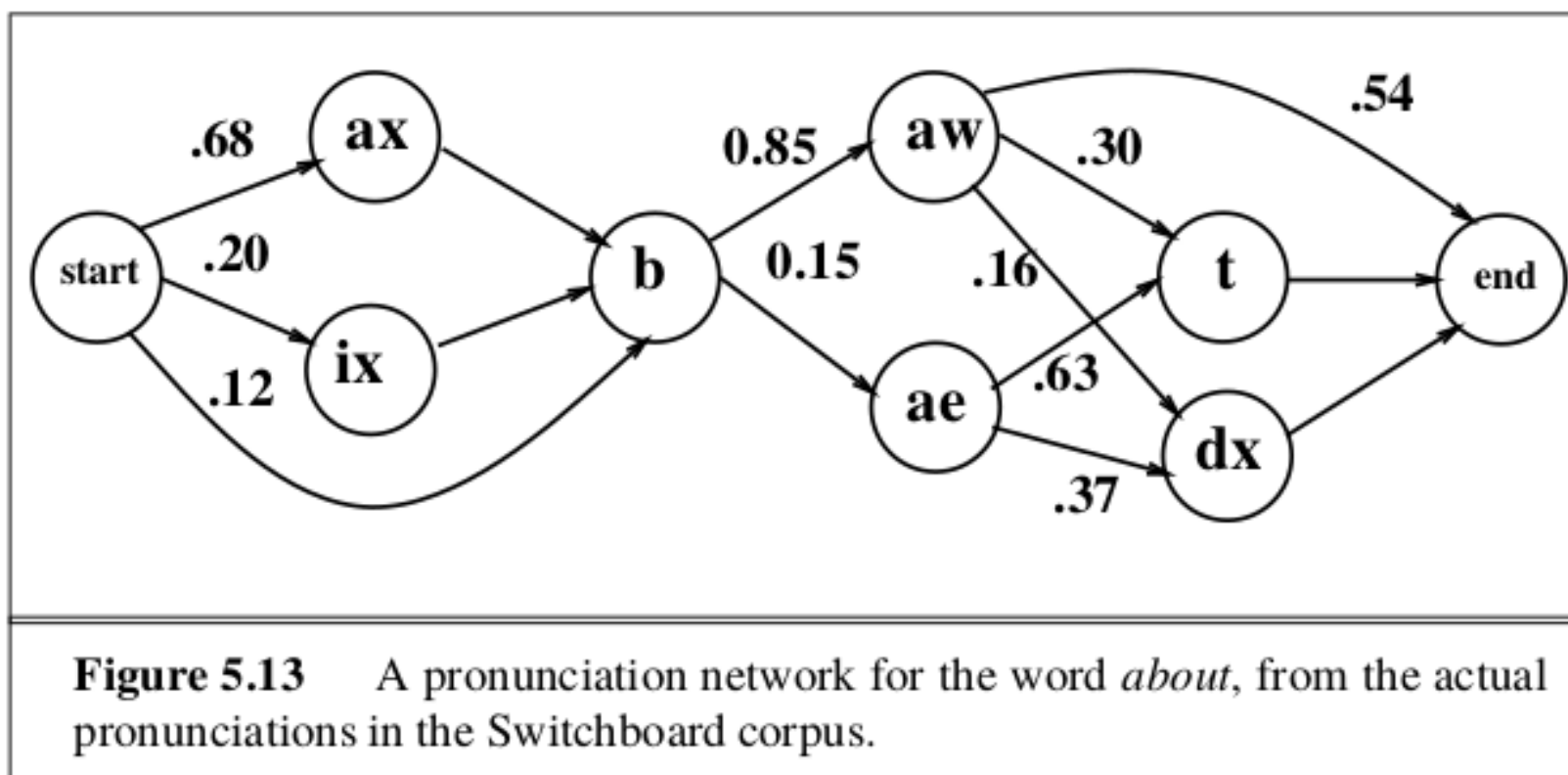


You say [t ow m ey t ow] and I say [t ow m aa t ow]. Two pronunciation networks for the word tomato. The top one models sociolinguistic variation (some British or eastern American dialects); the bottom one adds in coarticulatory effects.

- 
- First is to start with on-line pronunciation dictionaries and use hand-written rules
 - probabilities can then be assigned
 - either by counting the number of times each pronunciation occurs in a corpus,
 - or if the corpus is too sparse, by learning probabilities for each rule and multiplying out the rule probabilities for each surface form
 - Weighted rules automatically compiled into a weighted finite-state transducer

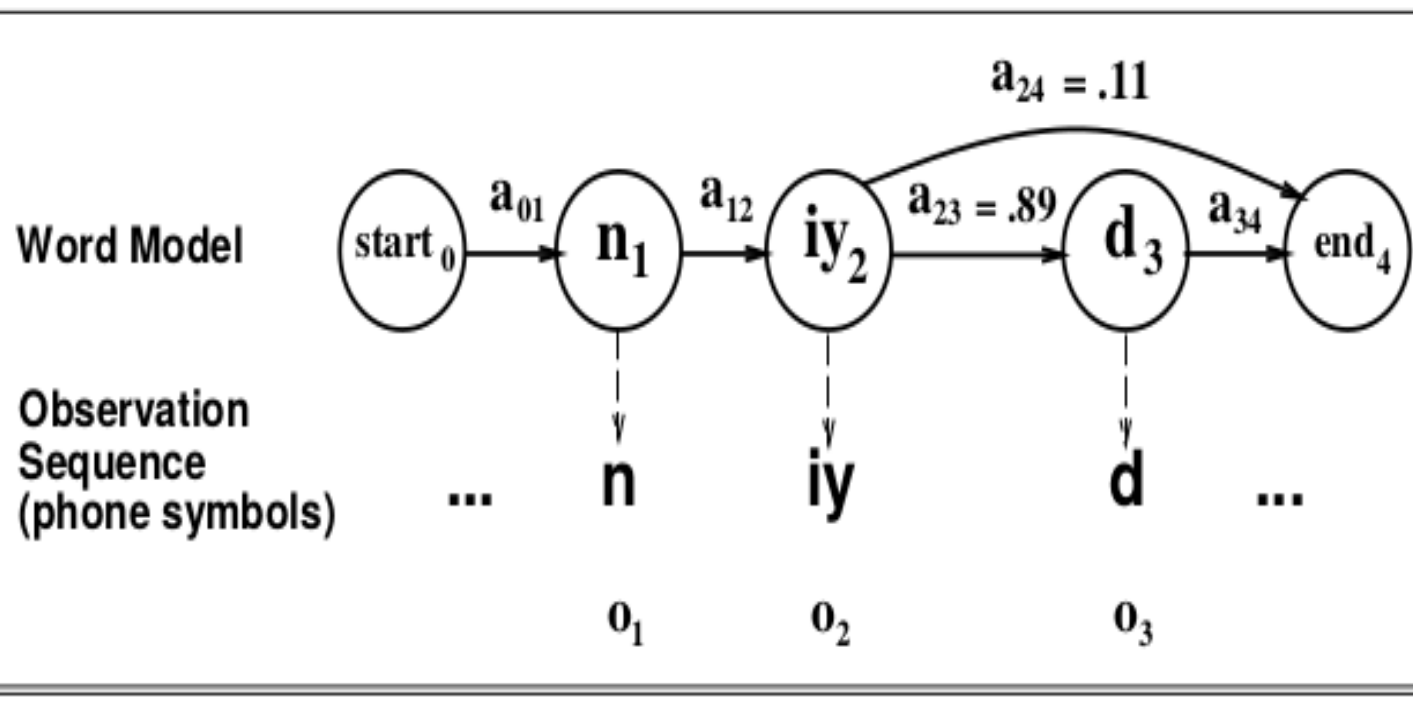
- 
- Alternatively, for very common words,
 - we can simply find enough examples of the pronunciation in a transcribed corpus to build the model by just combining all the pronunciations into a network.

A real world example



Computing Likelihoods from Weighted Automata: The Forward Algorithm

- A weighted automaton consists of
 - 1. a sequence of states $q = (q_0, q_1, q_2, \dots, q_n)$, each corresponding to a phone,
 - 2. a set of transition probabilities between states, a_{01} ; a_{12} ; a_{13} , encoding the probability of one phone following another.
- The states as nodes, and the transition probabilities as edges between nodes;
- an edge exists between two nodes if there is a non-zero transition probability between the two nodes
- Observation sequence
 - Sequence of symbols input to the model (Recognizer)
 - Sequence of symbols produced by the model (Generator)
- This task of determining which underlying word might have produced
- an observation sequence is called the **decoding** problem



Forward algorithm

- Simple algorithm to compute likelihood of a string of phones given weighted automaton for a word.
- The forward algorithm takes as input a pronunciation network for each candidate word
- Dynamic programming algorithm- slight generalization of MED

MED

- 1.Rows labelled by states
- 2.new value from 3 cells around it.
- 3.Minimum probability

Forward Algorithm

- 1.Rows labelled by state graphs
- 2.Complex recurrence relation
- 3.Sum of probabilities of all possible paths

- 
- Each cell of the forward algorithm matrix, $forward[t, j]$ represents the probability of being in state j after seeing the first t observations, given the automaton λ .

$$forward[t, j] = P(o_1, o_2 \dots o_t, q_t = j | \lambda) P(w)$$

1. The previous path probability from the previous cell $forward[t-1, i]$.
2. The transition probability a_{ij} from previous state i to current state j .
3. The observation likelihood b_{jt} that current state j matches observation symbol t . For the weighted automata that we consider here, b_{jt} is 1 if the observation symbol matches the state, and 0 otherwise.

function FORWARD(*observations, state-graph*) **returns** *forward-probability*

num-states $\leftarrow$ NUM-OF-STATES(*state-graph*)

num-obs $\leftarrow$ length(*observations*)

Create probability matrix *forward*[*num-states* + 2, *num-obs* + 2]

forward[0,0] $\leftarrow$ 1.0

for each time step *t* **from** 0 **to** *num-obs* **do**

for each state *s* **from** 0 **to** *num-states* **do**

for each transition *s'* from *s* specified by *state-graph*

forward[*s'*, *t*+1] $\leftarrow$ *forward*[*s*, *t*] * *a*[*s*, *s'*] * *b*[*s'*, *o*_{*t*}]

return the sum of the probabilities in the final column of *forward*

Figure 5.16 The forward algorithm for computing likelihood of observation sequence given a word model. $a[s, s']$ is the transition probability from current state *s* to next state *s'* and $b[s', o_t]$ is the observation likelihood of *s'* given *o*_{*t*}. For the weighted automata that we consider here, $b[s', o_t]$ is 1 if the observation symbol matches the state, and 0 otherwise.

need	end				.00056 * .11 = .00062
	d				
	iy			.00056 * 1.0 = .00056	
	n		.00056 * 1.0 = .00056		
	start	1.0			
		#	n	iy	#

Figure 5.17 The forward algorithm applied to the word *need*, computing the probability $P(O|w)P(w)$. While this example doesn't require the full power of the forward algorithm, we will see its use on more complex examples in Chapter 7.



Decoding : The viterbi Algorithm

Other names

DP alignment (for Dynamic Programming alignment),
dynamic time warping
one-pass decoding

Input

a single weighted automaton and a set of observed phones $O = (o_1, o_2, o_3, \dots, o_t)$

Output

The most probable state sequence $q = (q_1, q_2, q_3, \dots, q_t)$, together with its probability.



The algorithm first creates $N + 2$ or four state columns. The first column is an initial pseudo-observation, the second corresponds to the first observation phone $[n]$, the third to $[iy]$ and the fourth to a final pseudo-Observation.

The value $viterbi[t; j]$ is computed by taking the maximum over the extensions of all the paths that lead to the current cell

1. the **previous path probability** from the previous cell forward $[t-1; i]$.
2. **the transition probability** a_{ij} from previous state i to current state j .
3. the **observation likelihood** b_{jt} that current state j matches observation symbol t . Equals 1 if the observation symbol matches the state, and 0 otherwise.

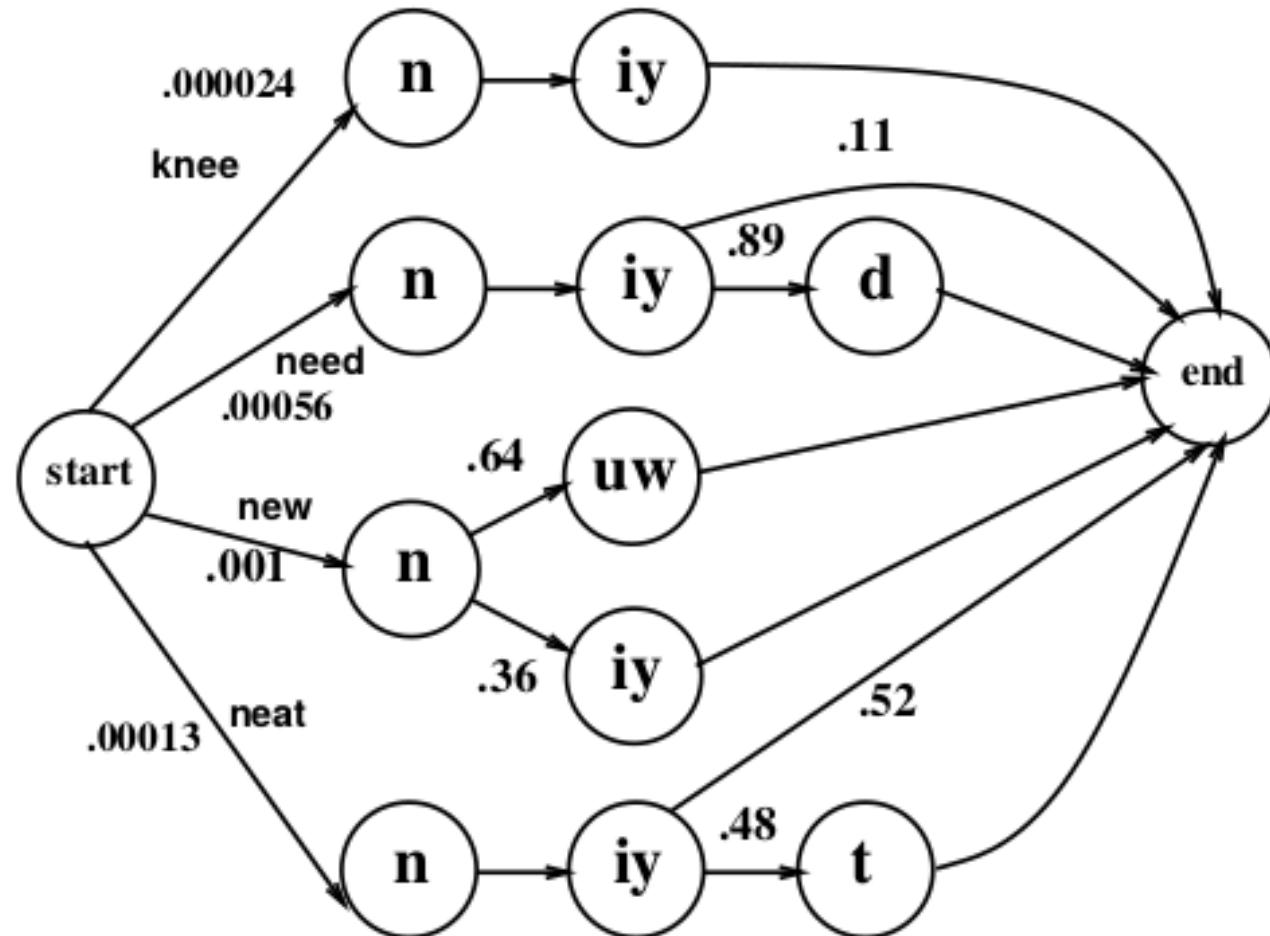


Figure 5.18 The pronunciation networks for the words *need*, *neat*, *new*, and *knee* combined into a single weighted automaton. Again, word probabilities are not usually considered part of the pronunciation network for a word; they are added here to simplify the exposition of the Viterbi algorithm.

function VITERBI(*observations* of len T , *state-graph*) **returns** *best-path*

$num-states \leftarrow \text{NUM-OF-STATES}(state-graph)$

Create a path probability matrix $viterbi[num-states+2, T+2]$

$viterbi[0,0] \leftarrow 1.0$

for each time step t **from** 0 **to** T **do**

for each state s **from** 0 **to** $num-states$ **do**

for each transition s' from s specified by *state-graph*

$new-score \leftarrow viterbi[s, t] * a[s, s'] * b_{s'}(o_t)$

if $((viterbi[s', t+1] = 0) \parallel (new-score > viterbi[s', t+1]))$

then

$viterbi[s', t+1] \leftarrow new-score$

$back-pointer[s', t+1] \leftarrow s$

Backtrace from highest probability state in the final column of $viterbi[]$ and return path

Figure 5.19 Viterbi algorithm for finding optimal sequence of states in continuous speech recognition, simplified by using phones as inputs. Given an observation sequence of phones and a weighted automaton (state graph), the algorithm returns the path through the automaton which has maximum probability and accepts the observation sequence. $a[s, s']$ is the transition probability from current state s to next state s' and $b[s', o_t]$ is the observation likelihood of s' given o_t . For the weighted automata that we consider here, $b[s', o_t]$ is 1 if the observation symbol matches the state, and 0 otherwise.

end					$.00036 * 1.0$ $= .00036$
neat	t				
	iy			$.00013 * 1.0$ $= .00013$	
	n	$1.0 * .00013$ $= .00013$			
need	d				
	iy			$.00056 * 1.0$ $= .00056$	
	n	$1.0 * .00056$ $= .00056$			
new	uw				
	iy			$.001 * .36$ $= .00036$	
	n	$1.0 * .001$ $= .001$			
knee	iy			$.000024 * 1.0$ $= .000024$	
	n	$1.0 * .000024$ $= .000024$			
start		1.0			
		#	n	iy	#

Figure 5.20 The entries in the individual state columns for the Viterbi algorithm. Each cell keeps the probability of the best path so far and a pointer to the previous cell along that path. Backtracing from the *end* state, we can reconstruct the state sequence $n_{new} \text{ } iy_{new}$, arriving at the best word *new*.